

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

ЛАБОРАТОРНАЯ РАБОТА

Исследование модели системы двумерной
биометрической аутентификации
Моделирование эмпирических двумерных
распределений случайных величин

по дисциплине «Введение в искусственный интеллект»

Вариант 2

Студент	ИУ8-13М (Группа)	Бочкарев Михаил Александрович (И. О. Фамилия) _____ (Подпись, дата)
Преподаватель		Басараб Михаил Алексеевич (И. О. Фамилия) _____ (Подпись, дата)

Оценка:

Москва, 2026 г.

СОДЕРЖАНИЕ

1	Цель работы	2
2	Постановка задачи	3
3	Исходные параметры моделирования	4
4	Алгоритмы моделирования	5
4.1	Треугольное распределение	5
4.2	Гауссово распределение	5
4.3	Формирование коррелированных параметров	5
5	Некоррелированные параметры	7
6	Коррелированные параметры	9
7	Декоррелирующее преобразование	11
8	Вывод	15

1 Цель работы

Исследовать синтетическую модель класса с использованием статистического моделирования, сформировать двумерные выборки для двух классов, оценить их эмпирические статистические характеристики и исследовать влияние корреляции параметров. Дополнительно выполнить синтез декоррелирующего преобразования для коррелированных биометрических параметров.

2 Постановка задачи

В соответствии с вариантом 2 требуется сформировать два класса объектов: “Свой” с треугольным законом распределения параметров и “Чужой” с гауссовым законом распределения. Объём каждого класса принимается равным

$$N = 1000.$$

Для каждого класса необходимо:

- 1) сформировать двумерную выборку с некоррелированными параметрами;
- 2) отобразить реализации обоих классов на одном графике;
- 3) вычислить центр масс, дисперсии, ковариационную и корреляционную матрицы;
- 4) получить коррелированные параметры путём поворота распределения на ненулевой угол и повторить расчёты;
- 5) для коррелированной выборки выполнить расчёты, необходимые для декоррелирующего преобразования, и проверить результат преобразования.

3 Исходные параметры моделирования

Для класса “Свой” выбраны математические ожидания

$$m_x = 2, \quad m_y = 4,$$

и среднеквадратичные отклонения

$$\sigma_x = 1, \quad \sigma_y = 0.4.$$

Для класса “Чужой” выбраны

$$m_x = -2, \quad m_y = 1,$$

$$\sigma_x = 1.2, \quad \sigma_y = 0.5.$$

Во всех случаях математические ожидания ненулевые и различаются, а отношение среднеквадратичных отклонений превышает 2 : 1.

4 Алгоритмы моделирования

4.1 Треугольное распределение

Для класса “Свой” используется симметричное треугольное распределение с модой, совпадающей с математическим ожиданием. Для заданного среднего m и среднеквадратичного отклонения σ границы распределения задаются как

$$a = m - \sqrt{6}\sigma, \quad b = m + \sqrt{6}\sigma, \quad c = m.$$

Генерация выполняется методом обратной функции распределения. Для равномерной случайной величины $U \sim U(0, 1)$ используется выражение

$$X = \begin{cases} a + \sqrt{U(b-a)(c-a)}, & U < 0.5, \\ b - \sqrt{(1-U)(b-a)(b-c)}, & U \geq 0.5. \end{cases}$$

4.2 Гауссово распределение

Для класса “Чужой” используется нормальное распределение. Стандартная нормальная случайная величина формируется преобразованием Бокса–Мюллера:

$$Z = \sqrt{-2 \ln U_1} \cos(2\pi U_2),$$

где U_1 и U_2 независимы и равномерно распределены на интервале $(0, 1)$. Далее выполняется масштабирование

$$X = m + \sigma Z.$$

4.3 Формирование коррелированных параметров

Для получения коррелированной выборки исходные двумерные точки поворачиваются на угол

$$\alpha = 35^\circ.$$

Матрица поворота имеет вид

$$A = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix}.$$

Поворот выполняется относительно центра соответствующего класса.

5 Некоррелированные параметры

На рисунке 1 показаны реализации обоих классов до поворота.

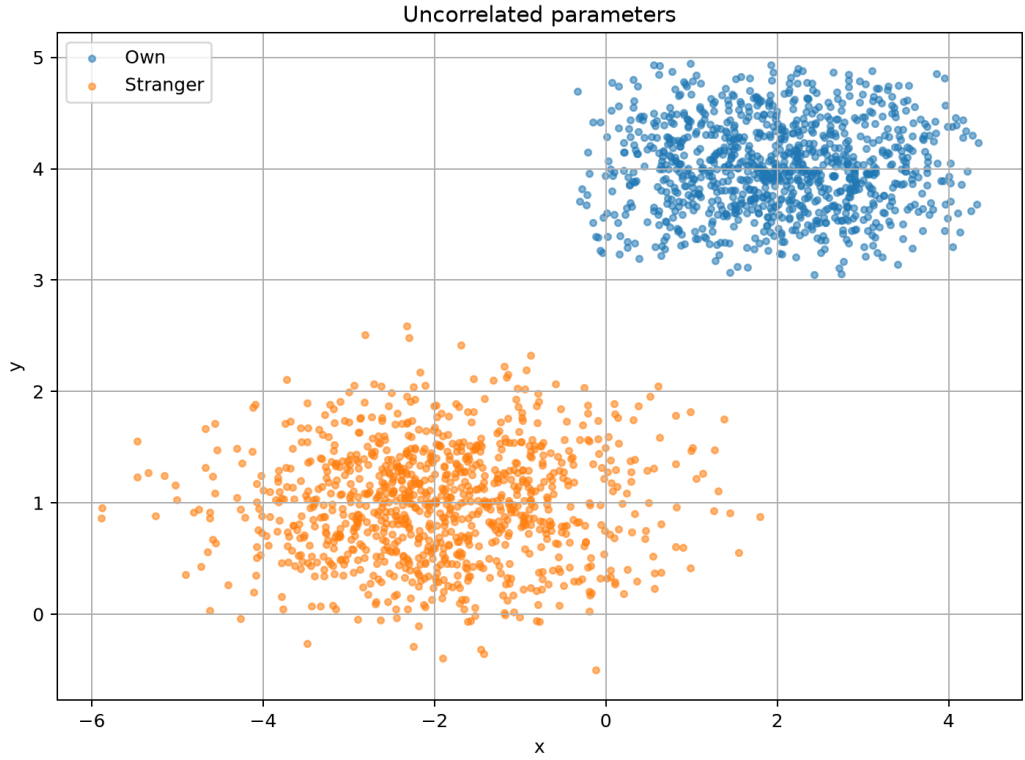


Рисунок 1 – Распределения классов при некоррелированных параметрах

Для класса “Свой” получены следующие оценки:

$$\bar{\mathbf{x}} = \begin{pmatrix} 1.98937 \\ 4.01525 \end{pmatrix},$$

$$\mathbf{D} = \begin{pmatrix} 1.01736 \\ 0.16019 \end{pmatrix},$$

$$\mathbf{R} = \begin{pmatrix} 1.01736 & 0.00399 \\ 0.00399 & 0.16019 \end{pmatrix},$$

$$\mathbf{Q} = \begin{pmatrix} 1.00000 & 0.00989 \\ 0.00989 & 1.00000 \end{pmatrix}.$$

Для класса “Чужой”:

$$\bar{\mathbf{x}} = \begin{pmatrix} -1.95651 \\ 0.99704 \end{pmatrix},$$

$$\mathbf{D} = \begin{pmatrix} 1.52268 \\ 0.25284 \end{pmatrix},$$

$$R = \begin{pmatrix} 1.52268 & 0.02177 \\ 0.02177 & 0.25284 \end{pmatrix},$$

$$Q = \begin{pmatrix} 1.00000 & 0.03508 \\ 0.03508 & 1.00000 \end{pmatrix}.$$

Внедиагональные элементы корреляционных матриц близки к нулю, что соответствует практически некоррелированным параметрам.

6 Коррелированные параметры

После поворота распределений на угол 35° получено расположение точек, представленное на рисунке 2.

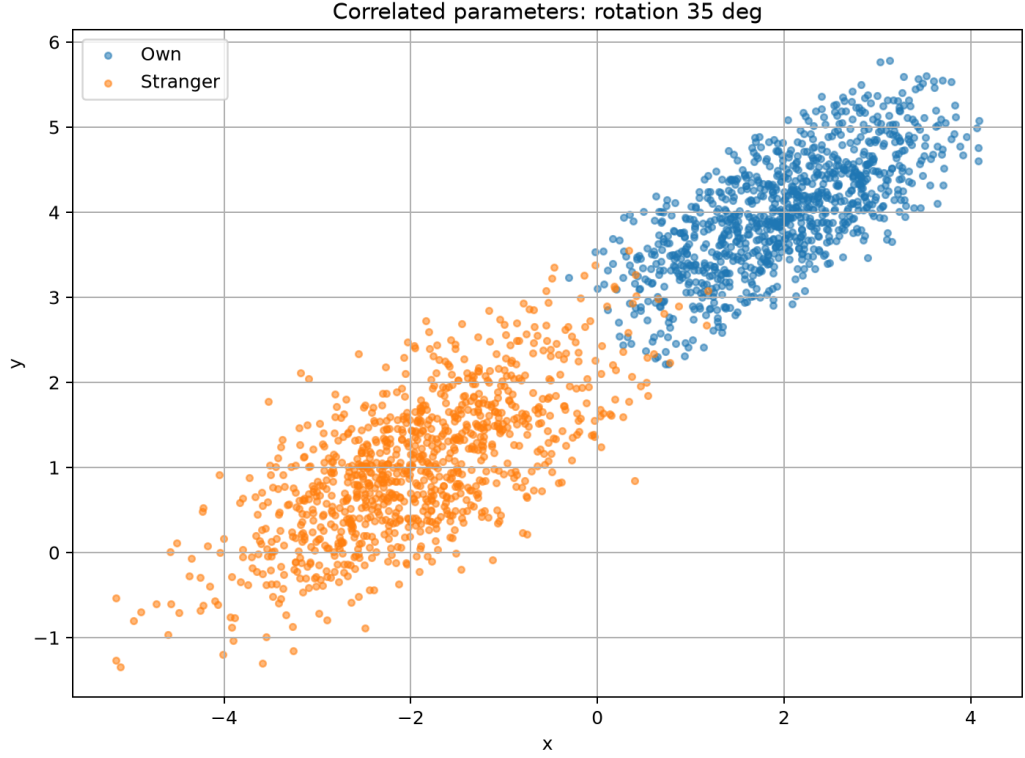


Рисунок 2 – Распределения классов после поворота

Для класса “Свой”:

$$\bar{\mathbf{x}} = \begin{pmatrix} 1.98255 \\ 4.00640 \end{pmatrix},$$

$$\mathbf{D} = \begin{pmatrix} 0.73161 \\ 0.44594 \end{pmatrix},$$

$$\mathbf{R} = \begin{pmatrix} 0.73161 & 0.40410 \\ 0.40410 & 0.44594 \end{pmatrix},$$

$$\mathbf{Q} = \begin{pmatrix} 1.00000 & 0.70748 \\ 0.70748 & 1.00000 \end{pmatrix}.$$

Для класса “Чужой”:

$$\bar{\mathbf{x}} = \begin{pmatrix} -1.96267 \\ 1.02252 \end{pmatrix},$$

$$\mathbf{D} = \begin{pmatrix} 1.08446 \\ 0.69106 \end{pmatrix},$$

$$R = \begin{pmatrix} 1.08446 & 0.60407 \\ 0.60407 & 0.69106 \end{pmatrix},$$

$$Q = \begin{pmatrix} 1.00000 & 0.69779 \\ 0.69779 & 1.00000 \end{pmatrix}.$$

После поворота абсолютные значения внедиагональных элементов корреляционных матриц увеличиваются. Это показывает появление статистической линейной связи между двумя параметрами.

7 Декоррелирующее преобразование

Для дополнительного исследования используется коррелированный класс “Свой”. Перед расчётом корреляционной матрицы выборка центрируется, а затем стандартизуется.

Ковариационная и корреляционная матрицы имеют вид

$$R = \begin{pmatrix} 0.73161 & 0.40410 \\ 0.40410 & 0.44594 \end{pmatrix},$$

$$Q = \begin{pmatrix} 1.00000 & 0.70748 \\ 0.70748 & 1.00000 \end{pmatrix}.$$

Обратные матрицы:

$$R^{-1} = \begin{pmatrix} 2.73659 & -2.47984 \\ -2.47984 & 4.48963 \end{pmatrix},$$

$$Q^{-1} = \begin{pmatrix} 2.00211 & -1.41645 \\ -1.41645 & 2.00211 \end{pmatrix}.$$

Собственные числа матрицы R :

$$\lambda(R) = \begin{pmatrix} 1.017376 \\ 0.160172 \end{pmatrix},$$

а собственные числа матрицы Q :

$$\lambda(Q) = \begin{pmatrix} 1.707479 \\ 0.292521 \end{pmatrix}.$$

Для обратных матриц получено

$$\lambda(R^{-1}) = \begin{pmatrix} 6.243297 \\ 0.982921 \end{pmatrix},$$

$$\lambda(Q^{-1}) = \begin{pmatrix} 3.418559 \\ 0.585659 \end{pmatrix}.$$

Числа обусловленности:

$$\text{cond}(R) = 6.35178, \quad \text{cond}(Q) = 5.83712,$$

$$\text{cond}(R^{-1}) = 6.35178, \quad \text{cond}(Q^{-1}) = 5.83712.$$

Нормированные собственные векторы ковариационной и корреляционной матриц образуют матрицы

$$D_R = \begin{pmatrix} -0.816472 & 0.577385 \\ -0.577385 & -0.816472 \end{pmatrix},$$

$$D_Q = \begin{pmatrix} 0.707107 & -0.707107 \\ 0.707107 & 0.707107 \end{pmatrix}.$$

Для корреляционной матрицы матрица декоррелирующего преобразования определяется как

$$D_Q^{-1} = \begin{pmatrix} 0.707107 & 0.707107 \\ -0.707107 & 0.707107 \end{pmatrix}.$$

Проверка спектрального разложения

$$Q = D_Q \Lambda_Q D_Q^{-1}$$

даёт

$$Q_{\text{восст.}} = \begin{pmatrix} 1.000000 & 0.707479 \\ 0.707479 & 1.000000 \end{pmatrix}.$$

Распределение стандартизованных параметров до преобразования показано на рисунке 3.

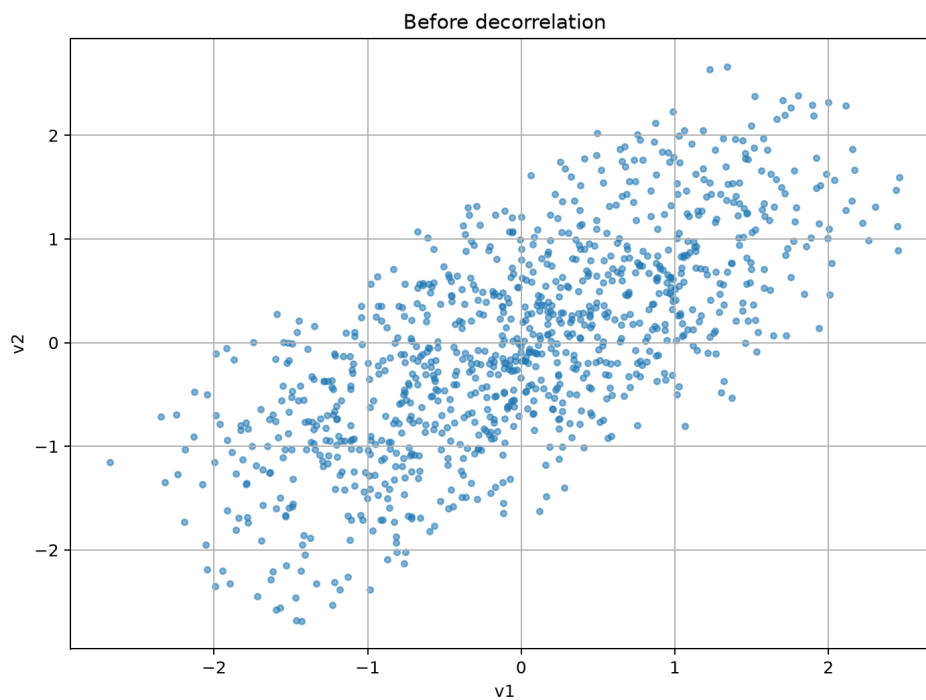


Рисунок 3 – Параметры до декоррелирующего преобразования

После применения преобразования $\mathbf{v}' = D_Q^{-1}\mathbf{v}$ получено распределение на рисунке 4.

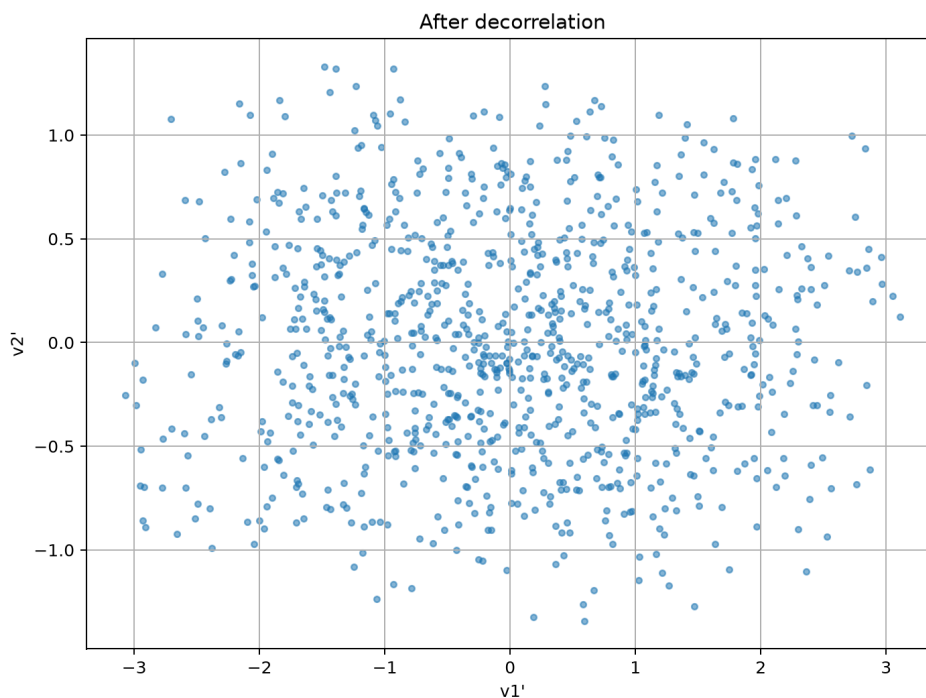


Рисунок 4 – Параметры после декоррелирующего преобразования

Ковариационная матрица преобразованных стандартизованных параметров:

$$R' = \begin{pmatrix} 1.707479 & -0.000000 \\ -0.000000 & 0.292521 \end{pmatrix}.$$

Корреляционная матрица:

$$Q' = \begin{pmatrix} 1.000000 & -0.000000 \\ -0.000000 & 1.000000 \end{pmatrix}.$$

Полученная корреляционная матрица практически диагональна, поэтому преобразованные параметры являются декоррелированными с точностью, определяемой конечным объёмом выборки и вычислительной погрешностью.

8 Вывод

В работе для варианта 2 были сформированы двумерные выборки классов “Свой” и “Чужой” с треугольным и гауссовым законами распределения соответственно. Для некоррелированных параметров рассчитаны эмпирические центры масс, дисперсии, ковариационные и корреляционные матрицы. Поворот распределений привёл к появлению выраженной корреляции между параметрами. Для коррелированного класса рассчитаны собственные числа и собственные векторы матриц корреляции и ковариации, их обратные матрицы и числа обусловленности. На основе собственных векторов корреляционной матрицы выполнено декоррелирующее преобразование, после которого внедиагональные элементы корреляционной матрицы стали близки к нулю.

ПРИЛОЖЕНИЕ 1 Программа моделирования

В листинге .1 приведена программа, использованная для формирования выборок, вычисления статистических характеристик, построения графиков и выполнения декоррелирующего преобразования.

Листинг .1 – Программа для варианта 2

```
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt

BASE = Path(__file__).resolve().parents[1]
FIGURES = BASE / "figures"
GENERATED = BASE / "generated"
FIGURES.mkdir(exist_ok=True)
GENERATED.mkdir(exist_ok=True)

N = 1000
SEED = 42
ANGLE_DEG = 35.0
rng = np.random.default_rng(SEED)

def triangular_symmetric(mean, std, size):
    """Symmetric triangular distribution generated by inverse CDF."""
    half_width = std * np.sqrt(6.0)
    a = mean - half_width
    b = mean + half_width
    c = mean
    u = rng.random(size)
    x = np.empty(size)
    left = u < 0.5
    x[left] = a + np.sqrt(u[left] * (b - a) * (c - a))
    x[~left] = b - np.sqrt((1.0 - u[~left]) * (b - a) * (b - c))
    return x

def gaussian_box_muller(mean, std, size):
    """Normal distribution generated by the Box-Muller transform."""
```

```

u1 = np.clip(rng.random(size), 1e-12, 1.0)
u2 = rng.random(size)
z = np.sqrt(-2.0 * np.log(u1)) * np.cos(2.0 * np.pi * u2)
return mean + std * z

def rotate(data, center, angle_deg):
    angle = np.deg2rad(angle_deg)
    matrix = np.array([
        [np.cos(angle), -np.sin(angle)],
        [np.sin(angle), np.cos(angle)],
    ])
    return (data - center) @ matrix.T + center

def statistics(data):
    mean = data.mean(axis=0)
    var = data.var(axis=0, ddof=1)
    cov = np.cov(data, rowvar=False, ddof=1)
    corr = np.corrcoef(data, rowvar=False)
    return mean, var, cov, corr

def latex_matrix(matrix, digits=5):
    rows = []
    for row in matrix:
        rows.append(" & ".join(f"{value:.{digits}f}" for value in row))
    return "\\begin{pmatrix}" + " \\\\".join(rows) + "\\end{pmatrix}"

def latex_vector(vector, digits=5):
    return latex_matrix(np.asarray(vector).reshape(-1, 1), digits)

def save_scatter(data_a, data_b, title, filename):
    plt.figure(figsize=(8, 6))
    plt.scatter(data_a[:, 0], data_a[:, 1], s=12, alpha=0.55, label="Own"
    )

```

```

plt.scatter(data_b[:, 0], data_b[:, 1], s=12, alpha=0.55, label="
    Stranger")
plt.xlabel("x")
plt.ylabel("y")
plt.title(title)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig(FIGURES / filename, dpi=180)
plt.close()

# Variant 2: Own = triangular, Stranger = Gaussian.
own_mean = np.array([2.0, 4.0])
own_std = np.array([1.0, 0.4])
stranger_mean = np.array([-2.0, 1.0])
stranger_std = np.array([1.2, 0.5])

own = np.column_stack([
    triangular_symmetric(own_mean[0], own_std[0], N),
    triangular_symmetric(own_mean[1], own_std[1], N),
])

stranger = np.column_stack([
    gaussian_box_muller(stranger_mean[0], stranger_std[0], N),
    gaussian_box_muller(stranger_mean[1], stranger_std[1], N),
])

own_unc = statistics(own)
stranger_unc = statistics(stranger)

save_scatter(
    own,
    stranger,
    "Uncorrelated parameters",
    "uncorrelated.png",
)

```

```

own_corr_data = rotate(own, own_mean, ANGLE_DEG)
stranger_corr_data = rotate(stranger, stranger_mean, ANGLE_DEG)
own_cor = statistics(own_corr_data)
stranger_cor = statistics(stranger_corr_data)

save_scatter(
    own_corr_data,
    stranger_corr_data,
    f"Correlated parameters: rotation {ANGLE_DEG:.0f} deg",
    "correlated.png",
)

# Auxiliary task: decorrelation of the correlated Own class.
centered = own_corr_data - own_corr_data.mean(axis=0)
std_sample = centered.std(axis=0, ddof=1)
standardized = centered / std_sample

R = np.cov(centered, rowvar=False, ddof=1)
Q = np.corrcoef(centered, rowvar=False)
R_inv = np.linalg.inv(R)
Q_inv = np.linalg.inv(Q)

lambda_R, D_R = np.linalg.eigh(R)
lambda_Q, D_Q = np.linalg.eigh(Q)
idx_R = np.argsort(lambda_R)[::-1]
idx_Q = np.argsort(lambda_Q)[::-1]
lambda_R = lambda_R[idx_R]
D_R = D_R[:, idx_R]
lambda_Q = lambda_Q[idx_Q]
D_Q = D_Q[:, idx_Q]

lambda_R_inv = np.linalg.eigvalsh(R_inv)[::-1]
lambda_Q_inv = np.linalg.eigvalsh(Q_inv)[::-1]

D_Q_inv = np.linalg.inv(D_Q)
Q_reconstructed = D_Q @ np.diag(lambda_Q) @ D_Q_inv

```

```

# Column-vector form:  $v' = D_Q^{-1} v$ . For row observations this is  $z @ (D_Q^{-1})^T$ .
decorrelated = standardized @ D_Q_inv.T
R_dec = np.cov(decorrelated, rowvar=False, ddof=1)
Q_dec = np.corrcoef(decorrelated, rowvar=False)

plt.figure(figsize=(8, 6))
plt.scatter(standardized[:, 0], standardized[:, 1], s=12, alpha=0.55)
plt.xlabel("v1")
plt.ylabel("v2")
plt.title("Before decorrelation")
plt.grid(True)
plt.tight_layout()
plt.savefig(FIGURES / "decorrelation_before.png", dpi=180)
plt.close()

plt.figure(figsize=(8, 6))
plt.scatter(decorrelated[:, 0], decorrelated[:, 1], s=12, alpha=0.55)
plt.xlabel("v1'")
plt.ylabel("v2'")
plt.title("After decorrelation")
plt.grid(True)
plt.tight_layout()
plt.savefig(FIGURES / "decorrelation_after.png", dpi=180)
plt.close()

results = f"""\% Auto-generated by code/task1_variant2.py
\\newcommand{{\\SampleSize}}{{{N}}}}
\\newcommand{{\\RotationAngle}}{{{ANGLE_DEG:.0f}}}

\\newcommand{{\\OwnMeanUnc}}{{{\\latex_vector(own_unc[0])}}}
\\newcommand{{\\OwnVarUnc}}{{{\\latex_vector(own_unc[1])}}}
\\newcommand{{\\OwnCovUnc}}{{{\\latex_matrix(own_unc[2])}}}
\\newcommand{{\\OwnCorrUnc}}{{{\\latex_matrix(own_unc[3])}}}

\\newcommand{{\\StrangerMeanUnc}}{{{\\latex_vector(stranger_unc[0])}}}
\\newcommand{{\\StrangerVarUnc}}{{{\\latex_vector(stranger_unc[1])}}}
\\newcommand{{\\StrangerCovUnc}}{{{\\latex_matrix(stranger_unc[2])}}}

```



```

Own, correlated:\nmean={own_cor[0]}\nvar={own_cor[1]}\ncov=\n{own_cor
    [2]}\ncorr=\n{own_cor[3]}\n
Stranger, correlated:\nmean={stranger_cor[0]}\nvar={stranger_cor[1]}\ncov
    =\n{stranger_cor[2]}\ncorr=\n{stranger_cor[3]}\n
Auxiliary decorrelation:\nR=\n{R}\nQ=\n{Q}\nR^-1=\n{R_inv}\nQ^-1=\n{Q_inv
    }\nlambda(R)={lambda_R}\nlambda(Q)={lambda_Q}\nlambda(R^-1)={
    lambda_R_inv}\nlambda(Q^-1)={lambda_Q_inv}\ncond(R)={np.linalg.cond(R)
    }\ncond(Q)={np.linalg.cond(Q)}\nD_Q=\n{D_Q}\nD_Q^-1=\n{D_Q_inv}\nQ
    reconstructed=\n{Q_reconstructed}\ncov after=\n{R_dec}\ncorr after=\n{
    Q_dec}\n""
(GENERATED / "results.txt").write_text(summary, encoding="utf-8")

print("Generated figures and generated/results.tex")

```